AGENTWARD + MUSEVERIFY

Evidence-driven security for Python, JavaScript and TypeScript

Project guide / current-stage edition

A Meta-primary harness that inspects source, reproduces supported vulnerabilities in Docker, proposes repairs and requires independent verification before accepting them.

3 languages 7 supported extensions	**7 phases** One mandatory gate chain	**Protocol v3** Deployed remote sandbox

Current release
Source-aware ingress guard, protected runtime evidence, persistent private connectivity and a pinned library of 818 engineering skills.

Evidence snapshot
78 classifier regression cases passed; 25 remote protocol-v3 acceptance groups passed. Their scope and dates are explained on page 14.

Application snapshot **59f8c1d**
31 October 2026

01 / PROJECT GUIDE

What the project is now

AgentWard + MuseVerify combines source inspection, controlled exploit reproduction and AI-assisted repair in a single evidence-driven workflow.

Purpose

The project helps an operator assess supported source files, distinguish reproducible vulnerabilities from inconclusive findings, and review candidate repairs with their supporting evidence. Model advice can propose a change; acceptance depends on the mandatory gates.

Area	Current stage
Input	Python plus single-file JavaScript/TypeScript Node modules. Language adapters select the parser, harness and verification rules.
Execution	Authenticated remote services and a deployed multilang-v3 Docker image; protected traces bind execution to the submitted artifacts.
Ingress	A pinned source-aware classifier replaces the older Prompt-Guard model. Parser facts precede the complete source; sanitization requires a full rescan.
Engineering	All 818 requested community cybersecurity skills are installed and pinned. Selected skills informed the hardening; runtime gates stay in application code.
Operations	Both services run on loopback behind persistent private Tailscale forwarding. Workstation integration was verified after SSH disconnected.

Read this guide by task

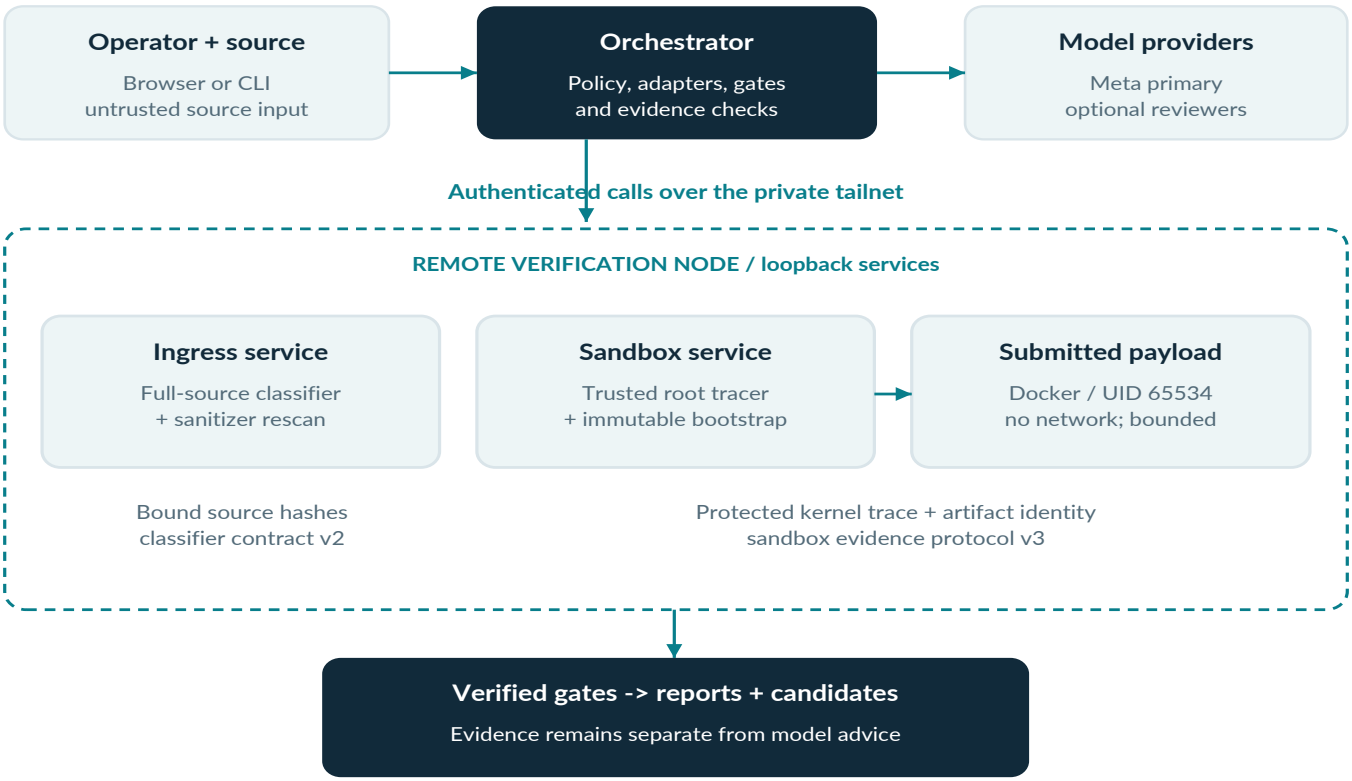
Task	Pages
Understand the architecture and supported inputs	3-5
Understand detection, evidence and repair acceptance	6-8
Run and operate the system	9-12
Review implementation, results and coverage limits	13-16

README.md; docs/SKILLS.md; docs/INGRESS_GUARD.md; docs/DEPLOYMENT_V3.md

02 / PROJECT GUIDE

Architecture and trust boundaries

The workstation coordinates policy and verification. Submitted source and model output remain untrusted throughout the workflow.



What must be trusted

The operator configuration, application services, selected provider identities, Docker host and kernel, runtime image, bootstrap and tracer form the trusted base. A compromised host or service can undermine the evidence it produces.

What evidence establishes

Authentication controls who can request work. Hashes bind results to particular source and harness artifacts. Protected kernel traces record selected execution behavior. The orchestrator combines that evidence with positive controls, reviews and regression results.

> **Two different contracts**
> **Classifier contract v2** binds model, policy, language and source hashes. **Sandbox protocol v3** binds protected runtime evidence, artifact hashes, language, runtime and filename. Both must match the workstation.

docs/THREAT_MODEL.md; README.md; service_security.py; sandbox_protocol.py

03 / PROJECT GUIDE

The seven-phase workflow

Every supported language uses the same orchestrator. A failed or unavailable required gate prevents a verified repair.

Phase	Work performed	Acceptance evidence
1 / Ingress Shield	Classify complete source and sanitize injected instructions.	Full coverage, rescan after changes, exact model/policy contract and source hashes.
2 / Syscall Sandbox	Reproduce supported exploits with deterministic probes in Docker.	Protected strace evidence, artifact identity and completed behavioral probes.
3 / Runtime Hooks	Apply screening, ensemble policy checks and trajectory monitoring.	Valid unanimous votes and the current operator policy revision; violations trigger rewind.
4 / Patch Verification	Synthesize a repair; inspect contracts, static paths, reviews, CodeShield and tripwires.	All applicable structural, scanner and independent-review gates pass.
4.5 / Adversarial Self-Play	Replay bounded mutated and multi-step scenarios.	Original-code positive controls succeed; no surviving exploit in the candidate; valid evidence.
5 / Regression	Replay exploits, normal inputs and decoy probes.	Exploit blocked; functional checks preserved; tripwire fires only for decoys.
5.1 / Semantic Hunter	Inspect related modules and semantic paths with Wonder-led analysis.	Static findings and unresolved assumptions remain visible, separate from runtime proof.

Concurrency and reporting

Phase 2 overlaps with graph construction and the initial policy baseline. The pipeline runs at most three containers concurrently. Passing that baseline cannot replace the later sandbox, patch or regression evidence.

Reports distinguish simulated outcomes, static findings and runtime-verified outcomes. Semantic hunting can identify additional review work beyond the behavior exercised by the repair probes.

README.md; main_pipeline.py; docs/LANGUAGES.md

04 / PROJECT GUIDE

Supported languages and targets

Language support means a defined parser, execution harness and verification contract. It does not imply arbitrary project builds.

Language	Files	Supported verification scope
Python	.py	Existing Python parsing, graph analysis and benchmark probes.
JavaScript	.js / .mjs / .cjs	Single-file Node ES/CommonJS modules with compatible exported SQL/command functions.
TypeScript	.ts / .mts / .cts	The same Node probes with erasable syntax. Original source executes in Node 24; scanning uses AST-based type erasure.

Node target contract

Node targets support named top-level functions and arrow functions in ES/CommonJS exports. All exports (maximum eight) must accept a single scalar SQL/command probe argument and return serializable results. Ambiguous or partial probe coverage is rejected.

Execution uses built-in Node dependencies. Normal regression uses **agentward_normal**; other business contracts need additional probes. Node always runs in Docker, including fixture-model demonstrations.

TypeScript behavior

Tree-sitter parses without execution. Type-only nodes are erased and reparsed as JavaScript for CodeShield. Original TypeScript executes in Docker. Supported syntax includes annotations, interfaces, type aliases, generics and as-assertions; there is no type checking.

Result	Meaning
UNSUPPORTED_LANGUAGE	Extension outside the registry: for example Go, Java, Rust, JSX or TSX.
INVALID_SOURCE	Syntax is invalid for the selected language.
UNSUPPORTED_TARGET	The supported language has no fully compatible Node probe surface.

Outside the current target contract
Browser apps, npm dependencies, multi-file execution, constructed classes and project compiler plugins are unsupported. TypeScript enums, namespaces, decorators and unsupported transformations cannot pass the complete chain.

docs/LANGUAGES.md; language_adapters.py; ecmascript_harness.py

05 / PROJECT GUIDE

Ingress: detection with context

The ingress guard detects instructions aimed at manipulating the audit. It is a separate task from finding vulnerabilities in the program.

How the current guard works

The **Horizon-Labs/prompt-injection-guard-small** model, pinned at revision **5ed691c** under Apache-2.0, receives bounded parser facts followed by the complete source. The prefix describes language, declared names and syntax-node kinds; it never declares the source safe. Valid syntax and filenames cannot bypass classification.

Control	Current behavior
Coverage	512-token windows with 128-token overlap; the highest injection score controls the decision. Risk threshold: 0.40.
Work bound	At most 256 windows per original or sanitized input. Over-limit or incomplete coverage is denied, without silently truncating source.
Sanitization	Recognizes explicit role-directed bypass or fabricated-audit instructions in comments/literals, including decoded base64. Modified source receives a full rescan.
Model integrity	Five artifacts are pinned and SHA-256 verified. Safetensors, no remote code, no inference-time downloads; provisioning is explicit.
Evidence	Contract v2 binds the model revision and policy identity, language and original/sanitized hashes. Stale or mismatched responses cannot authorize a live run.

What “detection remains probabilistic” means

The model assigns a learned risk score. It can occasionally flag harmless code or miss an unfamiliar instruction attack. **78 of 78 regression cases passed** means those specific cases behaved as expected; it does not mean every future input will be classified correctly.

The 0.40 threshold is a decision setting, not a measured 40% chance that a program is unsafe. Ingress ALLOW means the file can proceed to the next checks; a vulnerable SQL or command function may correctly pass ingress and then fail sandbox verification.

The older model produced the reported Node false positives. The replacement resolved those tested cases while retaining separate sandbox, review, scanner and regression requirements.

docs/INGRESS_GUARD.md; ingress_guard.py; security/ingress-model.lock.json

06 / PROJECT GUIDE

Protected runtime evidence

The remote sandbox executes submitted code under a fixed, constrained runtime and keeps the evidence outside the payload's control.

Boundary	Enforced control
Identity	Bearer authentication and bounded request schemas. Protocol v3 binds source/harness SHA-256, language, runtime and target filename.
Privileges	Trusted root tracer and immutable bootstrap start first. Submitted code runs only after UID/GID drop to 65534, group clearing and capability removal.
Filesystem / network	No container network. Read-only root, source inputs and bootstrap; bounded 64 MiB workspace and 16 MiB temporary storage.
Trace protection	Root-owned kernel trace in a POSIX-ACL-protected directory. Payloads cannot read, overwrite, unlink or replace host-backed evidence.
Resources	256 MiB memory/swap ceilings, CPU/PID limits and bounds on time, output, trace bytes and event counts. Overflow and timeout deny evidence and trigger cleanup.
Runtime choice	Fixed python3/node allowlist. Python isolated startup; Node loader environment variables cleared. No arbitrary caller-selected interpreter.

Conditions for accepted execution evidence

The client requires matching protocol and artifact identity plus complete protected traces and valid behavioral results. Missing traces, tampering, truncation, abnormal completion and unsupported ownership/ACL setup deny evidence. Docker failure cannot trigger production host execution.

Deployment requirements
The supported node uses rootful Linux Docker without user-namespace remapping and an ACL-capable temporary filesystem. Rootless or remapped configurations are not validated.

Evidence is authenticated and hash-bound, not digitally signed. Selected syscalls and payload-produced probe lines cannot prove all possible behavior; the other acceptance gates remain necessary.

docs/THREAT_MODEL.md; sandbox_server.py; sandbox_protocol.py

07 / PROJECT GUIDE

How a repair earns acceptance

A proposed patch stays a candidate until independent checks agree with the observed behavior.

Preserve the public contract

Patch checks preserve relevant function interfaces and sync/async behavior. Node checks also compare exported parameters and type signatures. Model-generated text cannot waive structural rules, scanning or current operator restrictions.

Use independent evidence at each step

Gate	What it contributes
Policy and review	Valid votes from the selected reviewers, bound to current additive operator constraints. Missing or malformed reviewer output denies approval.
Static checks and CodeShield	Contract inspection, call/taint findings and scanner results. TypeScript is erased and reparsed for JavaScript rules; execution uses the original source.
Structural tripwire	An AST-recognized decoy equality guard plus a logging action. Text that merely mentions a canary cannot satisfy this check.
Adversarial replay	Deterministic mutations and bounded scenarios run against original and candidate code. Original-code positive controls must reproduce the relevant behavior.
Regression	The original exploit is blocked, tested normal results are preserved, and the tripwire fires only for decoy probes.

Bounded self-play

Models propose scenarios in a bounded scenario language. The system uses deterministic harnesses for storage/replay, URL decoding, cleanup and supported interleaving checks. It does not execute arbitrary model-written self-play harnesses. Supported scenarios vary by adapter.

A storage adapter tests replay robustness; it may not demonstrate a native persistence flaw. Unsupported scenarios or failed positive controls cannot be counted as neutralized.

Keep candidates and evidence reviewable
Accepted candidates and exact diffs are written to **patches/<run-id>/**. Benchmark targets remain intentionally vulnerable. Additional semantic findings and coverage limits stay visible in the report.

README.md; museverify_stage4.py; main_pipeline.py; docs/LANGUAGES.md

08 / PROJECT GUIDE

Providers and operator control

Meta remains mandatory for every live run. Optional providers add independent review when they pass pre-flight.

muse-spark-1.3 leads runtime policy and patch synthesis. **muse-wonder-1.0** leads deeper semantic analysis. In Meta-only operation, the two native models cross-check each other.

Mode	Selected participants
auto	Meta primary plus every healthy secondary.
all_three	Meta, Anti-Gravity and Custom; notices identify unavailable secondary providers.
meta_plus_agy	Meta plus verified Anti-Gravity.
meta_plus_custom	Meta plus verified Custom BYOK.
meta_only	Spark and Wonder cross-check each other.

When no requested secondary passes pre-flight, Meta's other native model supplies the independent review. If Meta is offline, a live run cannot proceed. A selected reviewer that later fails cannot silently become an approval.

Pre-flight and configuration

Provider discovery runs concurrently and caches dashboard status for 60 seconds. A forced refresh reloads .env. Each run retains its successful discovery snapshot while configuration is unchanged; task failures still deny approval. Configure META_API_KEY, ENSEMBLE_MODE and optional Anti-Gravity or Custom endpoint/model credentials locally.

Steer the running session

```
POST /api/steer
{"constraint":"Forbid subprocess calls; enforce pathlib commonpath"}
```

Restrictions are additive and shared across batch workers. A monotonically increasing policy revision invalidates stale in-flight approval. Recognized subprocess/path restrictions have AST enforcement; broader natural-language restrictions also require model assessment.

> **Control remains with the operator**
> Trajectory monitoring can halt and rewind a session for policy violations. No UI control waives a mandatory gate. General natural-language steering is not a complete formal policy language.

README.md; ai_provider.py; agentward_stage3.py; dashboard.py

09 / PROJECT GUIDE

Start a supported audit

Use the local dashboard or CLI after configuring the providers and authenticated verification-node endpoints.

1 / Prepare the workstation

```
python3 -m venv venv
. venv/bin/activate
python -m pip install -r requirements-dev.txt
cp .env.example .env
```

Python 3.12+ is recommended. Populate the local .env with Meta credentials, the shared AGENTWARD_NODE_TOKEN, and AGENTWARD_INGRESS_URL / AGENTWARD_SANDBOX_URL. Keep secrets out of Git. The deployed private-node setup is described on the next page.

2 / Start the dashboard

```
python -m uvicorn dashboard:app --host 127.0.0.1 \
  --port 8080 --ws-max-size 1024 --ws-max-queue 4
```

Open http://127.0.0.1:8080. Inspect the real provider pre-flight matrix, select an ensemble and choose a benchmark or a source path within the workspace. Retest credentials when configuration changes.

3 / Run a known target

```
python main_pipeline.py benchmark_targets/cwe_78_cmd.js
python main_pipeline.py benchmark_targets/cwe_89_sqli.ts
python main_pipeline.py poisoned_app.py
```

The CLI and dashboard use the same language adapters and mandatory gates. Inspect each phase, the exact diff and the evidence-derived executive brief before integrating an accepted candidate.

> **A useful first comparison**
> **cwe_78_cmd.js** and **cwe_89_sqli.ts** exercise vulnerable command/SQL paths. **fp_node.mjs** is the benign Node control. Passing ingress is expected for all three: the sandbox then distinguishes the tested runtime behavior.

Fixture-mode demonstrations have SIMULATED labels and cannot export verified SARIF fixes. Any permitted Python host execution is test-only; Node still requires Docker. See README.md for the explicit offline-demo commands.

10 / PROJECT GUIDE

Deployed services and operations

The coordinated workstation, service and sandbox upgrade is complete in the recorded deployment. This guide describes that snapshot; it does not perform a new deployment.

Component	Recorded state
Application	Release 59f8c1d with the matching workstation and ingress contract.
Remote services	Ingress and sandbox systemd services enabled and active, bound to 127.0.0.1.
Private connection	Persistent Tailscale Serve forwards ports 8000/8001 to the loopback services. Connectivity was verified after SSH disconnected.
Sandbox image	agentward-sandbox:multilang-v3; recorded runtime versions Node 24.22.2 and Python 3.12.16, plus strace.
Authentication	Shared random node token in a root-owned mode-0600 node file and the ignored workstation .env. Execution/classification aliases require it.

Upgrade as a coordinated release

Deploy the full checkout and node requirements; provision and verify the pinned model; build the sandbox image; restart both services and the dashboard with matching contracts. deploy_remote_services.sh automates node setup and preserves an existing token. Do not mix a new client with stale service evidence.

Check the operational boundaries

Use each service's /health endpoint for connectivity, then authenticated ingress and execution controls for functional acceptance. A missing or short configured token produces HTTP 503; missing or incorrect request authorization produces 401. Exhausted classifier or sandbox slots produce 429 with Retry-After.

Keep the dashboard on loopback or behind authenticated access. Exact host/origin checks protect browser requests but do not authenticate native clients. Restrict tailnet access to the intended operators. Node requests disable redirects and ambient HTTP proxies.

Rollback and recovery
Pre-upgrade application, environment and service-unit backups were retained on the node. Restore a matching workstation/service/image set. An older unauthenticated node cannot serve as valid protocol-v3 verification. The repository deployment record contains the recovery details.

docs/DEPLOYMENT_V3.md; docs/INGRESS_GUARD.md; deploy_remote_services.sh

11 / PROJECT GUIDE

Dashboard, API and artifacts

The interface exposes progress and evidence without allowing advisory text to override a verified outcome.

Interface	Operator use
GET /api/health GET /api/languages	Inspect running state, phase identifiers, language extensions and adapter scope.
GET /api/providers POST /api/providers/mode	Inspect pre-flight and select an ensemble. force_refresh=true retests provider configuration.
POST /api/run POST /api/batch	Audit a supported workspace source path or run a bounded benchmark batch.
POST /api/steer POST /api/clean	Add a session restriction; prune temporary workspaces only when idle.
GET /api/metrics GET /api/executive-brief	Read outcomes, usage and timing, plus evidence, assumptions and graphs.
GET /api/sarif WebSocket /ws/telemetry	Export SARIF v2.1.0 or consume replayable events tagged by run.

Review surfaces

The dashboard includes seven phase cards, replayable telemetry, a unified/side-by-side diff viewer, an executive brief and a cross-file call/taint map. Advisory model commentary is labeled and cannot change the outcome established by the gates.

Artifact	Contents
patches/<run-id>/	Accepted candidates and exact diffs; original benchmarks stay unchanged.
reports/benchmark_metrics.json	Per-file results, reviewer votes, pre-flight state, usage and latency.
reports/agentward_sarif_report.sarif	Findings and eligible verified fixes in SARIF format.
reports/executive_brief.md / .html	Evidence-derived assessment with unresolved assumptions and coverage limits.

Reports are written atomically and archived per run. Runtime reports, local workspaces and credentials are excluded from Git. They are operator-writable records, not a signed append-only audit log.

12 / PROJECT GUIDE

Codebase and skill integration

The implementation separates orchestration, model access, language rules, service security and evidence contracts.

Module / area	Responsibility
main_pipeline.py / dashboard.py	Orchestration, CLI, session state, API and live telemetry.
ai_provider.py / agentward_stage3.py	Provider discovery, routing, runtime policy and trajectory checks.
language_adapters.py ecmascript_harness.py	Language registry, parsers, contracts, sanitization hooks and deterministic Node probes.
agentward_stage1.py / ingress_server.py ingress_guard.py / ingress_sanitizer.py	Ingress client/service, source-aware inference, sanitization and classifier evidence.
sandbox_server.py / sandbox_protocol.py service_security.py	Constrained execution, protected trace protocol, authentication and bounded schemas.
museverify_stage4.py / security_graph.py semantic_hunter.py / reporting.py	Patch gates, static paths, semantic findings and evidence-derived outputs.
security/ / systemd/ / tests/ / docs/	Pins, Dockerfile, service units, regression corpora, tests and architecture records.

818 pinned community cybersecurity skills

All skills from the requested independent community repository are installed locally for Codex. security/skills.lock.json pins source revision **54a7988** and full-tree hashes. Per-skill licenses and supporting material are preserved. The collection is not affiliated with Anthropic.

Tool-invocation security, container hardening, WebSocket testing, host-header testing and threat-modeling skills informed the controls documented here. This does not claim that every skill was executed or that its procedures establish compliance.

```
python3 tools/manage_cybersecurity_skills.py install
python3 tools/manage_cybersecurity_skills.py verify
```

Guidance has a defined role
The application does not load all 818 skills as runtime instructions or execute their scripts automatically. Installing them does not add language adapters or authorize testing outside the operator's requested scope.

docs/SKILLS.md; security/skills.lock.json; repository modules at 59f8c1d

13 / PROJECT GUIDE

What has been verified

Evidence reviewed for application release 59f8c1d on 31 October 2026. Different checks establish different parts of the system.

Evidence set	Recorded result	What the result supports
Automated release checks	**92 tests**	91-test full suite passed with real Docker and no skips; an additional corpus-validity test passed. GitHub CI passed for the release.
Real ingress model	**78 / 78 cases** 45 benign + 33 malicious	Benign inputs stayed unchanged and passed. Malicious instructions were blocked or sanitized and successfully rescanned.
Remote protocol-v3 deployment	**25 acceptance groups**	Authenticated service aliases, all seven extensions, artifact/language binding, real isolation and exploit/fixture-repair controls. Recorded before the later classifier replacement.
Latest live integration	**4 benchmark paths** + 2 syntax controls	After guard deployment: real ingress and Docker for Python/JS command, TS SQL and benign Node. Additional TS object-literal and CommonJS source passed ingress.
Earlier browser acceptance	**JS/TS paths passed**	Simulated runs through the source-path UI, unsupported-language handling and zero page errors. This was not a live provider acceptance run.

How to read the classifier result

The 78 cases include the reported false positives, normal object literals and exports, multilingual instructions, base64, long inputs, fake metadata and repeated documentation. These corpora were used during policy development. The result is regression evidence, not an accuracy estimate from an untouched holdout population.

> **The current live acceptance boundary**
> The latest checks used the real classifier and real Docker. Repair checks used deterministic fixtures. They did **not** repeat a full live, model-generated seven-phase remediation run or revalidate current provider availability.

The 30 October live-provider record belongs to an older application snapshot. It is retained as history in docs/VERIFICATION.md and is not presented as current acceptance.

docs/INGRESS_GUARD.md; docs/DEPLOYMENT_V3.md; local release records; CI

14 / PROJECT GUIDE

Interpreting results and limits

Use each outcome as evidence for the behavior and coverage actually checked. A completed run is not a formal proof of program safety.

Result or signal	Interpretation
Ingress ALLOW / SANITIZED	The source may continue to the remaining gates. This is not a vulnerability verdict.
Ingress BLOCKED / UNVERIFIED	A classified attack was denied, or required evidence was absent/stale. The pipeline records an ingress-blocked outcome when live ingress cannot pass.
REMEDIATED	Applicable gates and tested repair behavior passed with production evidence. Review the diff and stated assumptions before integrating it.
FALSE_POSITIVE	The tested suspected vulnerability was not confirmed and the required safe-path evidence passed. This is distinct from a classifier false positive.
SIMULATED_*	Fixture-based demonstration. It cannot establish a production repair or export a verified SARIF fix.
INCONCLUSIVE / rejected / failed	Evidence, policy, repair or regression requirements were not satisfied. Missing evidence is never an implicit pass.

Coverage that remains bounded

Detection: unfamiliar instructions can evade the learned model; benign code can still be misclassified. New regressions should expand the corpus and trigger review of the pinned policy and model.

Program analysis: Python propagation is bounded; Node taint analysis is intraprocedural. Reflection, closures, dynamic dispatch, external libraries and Node cross-file flows need additional review. CodeShield coverage is narrower than a complete security audit.

Runtime and operations: selected probes and syscall traces cover observed behavior. The kernel, daemon, service, tracer and operator remain trusted. The shared token identifies an authorized workstation, not an individual user; provider data handling remains an operator decision.

A useful next validation step
For a new production target, add representative functional probes and run the full live pipeline with fresh provider pre-flight. Expand adapters explicitly for new languages, application shapes and vulnerability families. These are extension directions, not shipped capabilities.

docs/THREAT_MODEL.md; docs/LANGUAGES.md; docs/INGRESS_GUARD.md

15 / PROJECT GUIDE

References and reproducibility

The PDF is a reviewed snapshot. Versioned repository documents and their implementation provide the detailed source of each claim.

Reference	Use it for
README.md	Setup, provider configuration, API routes and complete verification commands.
docs/INGRESS_GUARD.md	Classifier policy, model pin, regression evaluation and deployment.
docs/THREAT_MODEL.md	Trust boundaries, threat inventory and residual limits.
docs/LANGUAGES.md	Adapter contracts, syntax and execution limits.
docs/DEPLOYMENT_V3.md	Recorded remote acceptance and coordinated rollback.
docs/SKILLS.md	Community provenance, exact pin and applied skills.

Release CI run provides the automated release record. Detailed live acceptance JSON remains in ignored local reports; those files are not part of the published guide.

Reproduce checks and rebuild this guide

```
python -m unittest discover -v
python -m pip check
python tools/manage_ingress_model.py verify
python tools/evaluate_ingress_guard.py \
  --output reports/ingress-regression.json
```

The model commands require the provisioned node dependencies and pinned artifacts. docs/INGRESS_GUARD.md lists the two additional corpora needed to reproduce all 78 cases; README.md lists the real Docker and live E2E checks.

The maintained PDF source is **tools/build_project_guide.py**. Install **requirements-docs.txt**, then run the generator. output/pdf/README.md records the build and review process. Rebuilding the PDF does not query services or establish new acceptance.

Terms: AST: abstract syntax tree. BYOK: bring your own API key. CWE: Common Weakness Enumeration. SARIF: Static Analysis Results Interchange Format. Positive control: an original-code test that must reproduce the expected vulnerable behavior.

Application revision: 59f8c1dac5cfde863d4e9ed62303b9f9e09ffc73